File Assembly and Collision Resolution Guide

A dedicated monograph on the decoder's most involved part — assembling a file from accumulated sectors and resolving version collisions. The document complements `AlgorithmGuide.en.md` (whose section 9 gives an overview): here the same topic is treated formally — the problem statement, data structures and their invariants, step-by-step algorithms, the mathematics of exhaustive search, heuristic rotation, and volume subset search, worked examples, and the exact boundaries of the guarantees. Familiarity with the context (packet format, the H5/D3 hashes, accumulation) is assumed. An engineering retelling of the same material with matching section numbering is `AssemblyGuide.Engineer.en.md`; a from-scratch tutorial that spells out every step with numeric traces is `AssemblyGuide.Tutorial.en.md`.

1. Problem Statement

Let a reception slot (`ReceptionSlot`) be created by a header with the fields:

- `N` — the number of data volumes, `M` — the number of ECC volumes, `T = N + M`;
- `FileSize` — the file size in bytes;
- `H3` — the SHA-256 of the original file contents (32 bytes);
- `H5` — the header hash, the seed of the sector hash.

By assembly time the accumulator is in the state

```
for every number i ∈ [0, T):  V_i = {(payload_1, c_1), …, (payload_k, c_k)}
```

— the set of payload **versions** of sector `i` with confirmation counters `c_j ≥ 1`; for absent sectors `V_i = ∅`.

The assembly problem. Find a selection `s: [0, T) → payload ∪ {⊥}` (`⊥` — volume not received) such that

1. all of the first `N` volumes are defined directly **or** recoverable by ECC from the selection;
2. `SHA-256( trim( s[0] ‖ s[1] ‖ … ‖ s[N-1], FileSize ) ) = H3`.

The arbiter — the hash of the whole file — **does not depend on the reception process**: neither the counters, nor the arrival order, nor heuristic decisions affect the final check. Therefore the outcome is binary: either the file, bit-for-bit equal to the original (proof: the SHA-256 match), or a refusal (`null`). "Partially correct" results do not exist by construction.

2. Collision Origins and the Threat Model

A **version collision** is the state `|V_i| > 1`: two or more different payloads accumulated under one sector number, each passing `D3 = Trunc9(SHA-256(H5 ‖ D1 ‖ D2))`.

Collision sources:

Source	Probability/condition	Consequence
Random hash collision	~2^{-72} per "candidate payload, known H5" pair	Extremely unlikely, but finite
Deliberate forgery	H5 travels openly in the stream; the attacker generates a valid sector with a foreign payload	Feasible; this is the designed threat model
Duplicates and repeats	Payload matches → the same version's counter grows	**Not** a collision

A forgery cannot pass the final check (that would need a SHA-256 preimage under H3), so its best outcome is an assembly refusal (DoS) or wasted search time. The defense is layered:

1. **Confirmation counters** — legitimate repeats strengthen the true version; for a forgery to win it needs repeats of its own;
2. **Combination search** — when versions are equally plausible, the search/rotation checks the candidates;
3. **The whole-file SHA-256** — the absolute arbiter.

3. Data Structures and Invariants

Type	Role
`SectorVariant`	payload (64 B) + `ConfirmationCount` (int, saturating at int.MaxValue)
`List<SectorVariant>`	the versions of one number; invariant — non-increasing counters
`SortedDictionary<int, List<SectorVariant>>`	the slot state; the key is the sector number
`ChoicePoint`	sector number + a **live reference** to the version list + `TiedVariantCount`
`SectorVersionInfo`	a public version snapshot (a payload copy) for UI/diagnostics

Version-list invariants:

- **I1 (sorting).** Counters are non-increasing by index: `c_1 ≥ c_2 ≥ …`. Maintained by the reception algorithm (section 4).
- **I2 (stability of equals).** Versions with equal counters keep their current mutual order; no operation swaps them with each other. The order may only change by the rotation of the equally-probable head (section 11) — again without breaking I1.

The **live reference** `ChoicePoint.Variants` is not a copy but the slot's own list: heuristic rotation mutates the version order directly in the slot, so building the selection after a rotation observes the new state. Per-attempt snapshots are not taken — their cost in the hot search loop is unjustified.

4. The Version Reception Algorithm (`AddSector`)

Input: number `i`, payload `p` (64 B). Steps:

1. **Validation.** `i ∉ [0, T)` or `|p| ≠ 64` → reject (false recognition is excluded by the hash, but the check is cheap and protects the API).
2. **First version of the number.** `V_i = ∅` → create the list, add `(p, 1)`. Done.
3. **Match search.** A linear pass over `V_i` with byte-wise payload comparison:
   - **version `(p, c)` found** → `c ← min(c + 1, int.MaxValue)` (saturation), then **bubbling up**: while the neighbor above has a *strictly* smaller counter, swap. Never swapped past equal counters — this preserves I2. Done.
   - **not found** → append `(p, 1)` **at the end** of the list. Correctness: all existing versions have `c ≥ 1`, the new one exactly 1, so I1 holds.

Complexity — `O(|V_i|)` per received copy; `|V_i| = 1` in the overwhelming majority of cases, which is why no payload hash table is used (a 64-byte comparison is cheaper than an extra structure and memory).
A side effect of bubbling: a version that gained another confirmation overtakes less confirmed ones — the list head `versions[0]` is always the most confirmed version. The assembly's "preferred selection" rests exactly on this.

5. The Assembly Pipeline: Overall Plan

```
TryAssemble
    1. selected ← BuildPreferredSelection()        // versions[0] of every number
    2. points   ← BuildChoicePoints()              // numbers with an equally-probable
  head
    3. points = ∅ ?
         yes → a single attempt: direct assembly → RS    // sections 6-7
         no ↓
    4. C ← Π T_i                                   // saturating at long.MaxValue
    5. attempt #0 (the zero combination) timed, duration t₀
    6. exhaustive-search estimate t^= t₀ · C
    7. C ≤ MaxExhaustiveCombinations (100,000) and t^≤ TimeBudget (30 s) ?
          yes → exhaustive search (section 10)
          no  → heuristic rotation (section 11)
    8. result = ⊥ and M > 0 → volume subset search  // section 12
```

Both search paths perform the **same** single attempt on every iteration (section 6 → section 7), differing only in how the next combination is generated. Cancellation (`CancellationToken`) and the time budget are checked between attempts. Stage 3 fires last, when the version choice is exhausted (or there were no choice points at all): the search space switches from versions to erasure maps — details in section 12.

6. A Single Attempt: Direct Assembly

```
TryAssembleDirect(selected):
    1. if ∃ i < N: selected[i] = ⊥        → fail (not all data on hand)
    2. buffer ← concat(selected[0..N))     // exactly N · 64 bytes
    3. TrimAndVerify(buffer)               // section 13
```

Complexity — `O(N · 64)` for concatenation plus one SHA-256 over `FileSize` bytes. The direct path covers the complete or over-complete reception case (losses covered by repeats rather than ECC).

7. A Single Attempt: RS Recovery

Direct assembly failed and `M > 0`:

1. Build a validity map `map[0..T)` (`true` — the volume is present in the selection: `selected[i] ≠ ⊥`) and the slot array with the selected payloads.
2. If fewer than `N` volumes are present, there is nothing to recover from — fail.
3. `RsCodecAdapter.Decode(sectors, map, N)`:
   - all data volumes intact → **passthrough** (returned as is);
   - erased data volumes outnumber the available ECC volumes → fail (the code falls short);
   - otherwise — solving the system over GF(2^{16}) with the erasure map: 32 independent symbol positions, each a column of `T` UInt16 symbols; `Process` recovers the erased data symbols; the recovered volumes are assembled from the recovered columns.
4. Validate the result: exactly `N` volumes of 64 bytes; concatenate `N · 64` → `TrimAndVerify` (section 13).

The key circumstance: **ECC volumes take part in the selection on par with data**, so a collision may sit on an ECC number too — a forged ECC volume corrupts recovery, the final hash mismatches, and the combination search must vary ECC versions as well. RS recovery is embedded in every attempt: the search varies only the ambiguous slots while the holes are re-covered by ECC each time. Stage 3 (section 12) uses the same procedure — with the difference that alongside the missing volumes the map receives **forcibly excluded** ones (`forcedErasure`): RS must recover those too.

8. Choice Points and the Search Space

A **choice point** (`ChoicePoint`) is a number `i` whose maximum counter is reached by more than one version:

```
T_i = |{ v ∈ V_i : c(v) = max_{u ∈ V_i} c(u) }| ≥ 2
```

`BuildChoicePoints` counts the length of the contiguous head of the list whose counter equals `versions[0].ConfirmationCount`.

The search space is the Cartesian product of the equally-probable heads:

```
Search = { (v_1, …, v_m) : v_j ∈ the first T_j versions of slot j }
  |Search| = C = Π T_j
```

A documented limitation: versions with a **strictly smaller** counter take part in neither search strategy (the odometer and the rotation operate on the head `T_j` only). If the true payload ends up in the confirmation minority (the forgery duplicated more often than the truth), the current assembly call ends in refusal. This is a deliberate trade-off:

- the probability of a hash-valid forgery is ~2^{-72} per packet; a forgery "winning" additionally requires a confirmation majority;
- continued reception changes the counters — the next `TryAssemble` call sees a new configuration;
- the sector may not be needed at all: RS will recover it.

9. Counting Combinations and Choosing the Strategy

`CountCombinations(factors)` is the product of the `T_j` in `long` with **saturation**: on anticipated overflow it returns `long.MaxValue` (reported as "> long.Max"). Saturation guarantees a correct comparison against the limit without arithmetic exceptions. `ShouldUseExhaustiveSearch` requires **both**:

1. `C ≤ MaxExhaustiveCombinations` — a hard combinatorial limit (100,000 by default);
2. `t₀ · C ≤ TimeBudget` — an empirical estimate: the zero attempt is timed (`Stopwatch`), its duration multiplied by `C`. A single attempt includes SHA-256 and possibly RS — the cost depends on the file size, so a static estimate does not suffice.

Violating either condition switches the search to heuristic rotation.

10. Exhaustive Search (the Odometer)

The selection indexes live in `indexes[0..m)`, the moduli in `moduli[j] = T_j`. `AdvanceIndexes` is a classic **odometer**: the lowest position on the right; increment with carry; an overflow of the highest position means the combinations are exhausted (`false` returned, indexes reset).

Pseudocode:

```
TryExhaustiveSearch:
    attempt #0 has already been run by the caller
    for completed = 1 .. C-1:
        if the cancellation token fired   → exception
        if the timer exceeded TimeBudget  → fail
        AdvanceIndexes(indexes, moduli)   // the next combination
        ApplyIndexes(selected, indexes)   // substitute the choice-point heads
        result = a single attempt         // sections 6-7
        if result ≠ ⊥ → return it
    fail
```

`ApplyIndexes` writes `point.Variants[indexes[j]].Payload` into `selected[point.SectorNumber]` — only the local selection is mutated; the slot's version order is untouched.
An odometer trace for `moduli = (2, 3)` (indexes left to right — highest, lowest):

```
(0,0) → (0,1) → (0,2) → (1,0) → (1,1) → (1,2) → reset
```

The exhaustive search is **complete** within the search space: with `C ≤ 100,000` and the budget met, the correct combination (if it lies in the heads `T_j`) is found guaranteed.

11. Heuristic Rotation

When exhaustive search is unreasonably expensive, an observation about the structure of a typical forgery is used: the attacker duplicates the forgery in all attacked slots **identically**, so the rank displacement of the true versions is systematic too. Rotation checks the "diagonals" of the space.

Algorithm:

```
TryRotationSearch:
    stateCount ← LCM(T_1, …, T_m), capped at MaxHeuristicAttempts (100,000)
    for state = 1 .. stateCount-1:
        RotateTiedVariants(points)    // in every slot the first equally-probable
                                      // variant moves to the end of the head T_j
        if budget/cancellation        → fail
        selected ← BuildPreferredSelection()  // the slots' new state
        result = a single attempt
        if result ≠ ⊥ → return it
    fail
```

Formal analysis. After `k` rotations the selected version of slot `j` has original index `k mod T_j` (the head is cyclically shifted by `k`). Hence attempt number `k` tests the combination

```
(k mod T_1, k mod T_2, …, k mod T_m)
```

— a "diagonal" of the space. Therefore:

- The **period** of the attempt sequence is `LCM(T_1, …, T_m)`; that many unique states exist (`CountRotationStates`, capped at `MaxHeuristicAttempts`).
- **Reachability.** A combination `(a_1, …, a_m)` is reachable ⟺ there exists `k` with `k ≡ a_j (mod T_j)` for all `j` — by the generalized Chinese remainder theorem ⟺ `a_i ≡ a_j (mod gcd(T_i, T_j))` for all pairs. For pairwise-coprime `T_j` the LCM equals C = Π T_j and rotation is **equivalent to exhaustive search** (in a different traversal order). With common divisors some combinations are unreachable — the price of the heuristic.
- A **systematic forgery** (identical displacement in all attacked slots: the `a_j` are equal) lies on the diagonal `k = a` — covered by one of the first attempts.

Example `T = (3, 2)`, LCM = 6 — all 6 combinations reachable:

```
k:      0      1      2      3      4      5
  pairs: A+X → B+Y → C+X → A+Y → B+X → C+Y   (then repeats)
```

Example `T = (2, 2, 2)`, C = 8, LCM = 2 — only `(0,0,0)` and `(1,1,1)` are reachable: rotation checks 2 of the 8 states, but both "systematic" diagonals — in the first and second attempts.

The slot-state mutation by rotation is safe for the invariants: moving within the equally-probable head changes neither the multiset of counters nor their order (I1); the mutual order of equal-counter versions changes only inside the head (I2 in its extended form). Consequences for repeated calls: the head order is "rotated", but assembly does not depend on that — any permutation of an equally-probable head is an equally valid starting point.

12. Volume Subset Search (Stage 3)

Stages 1–2 are powerless against **silent corruption**: a sector is damaged, yet its number is intact and the truncated D3 hash matches — a random corruption happened to pass the 72-bit check (probability ~2^{-72}) or the payload was crafted together with its hash (a forgery); the version is the only one. There is no choice point; RS with the full map takes the corrupted volume for the truth; every attempt honestly returns `null`. Stage 3 changes the search space: what varies is not the version choice but the **erasure map**. Formally, fix the sets `D` (missing data volumes), `A` (received ECC volumes), and an **exclusion mask** `X ⊆ [0, T)`:

```
map[i] = (volume i received ∧ i ∉ X)
  feasibility: 0 < |D| + |X ∩ data| ≤ min(|A| - |X ∩ ECC|, MaxErasedDataVolumes)
  attempt(X) = RS-Decode(map) → TrimAndVerify      // section 13
```

The first condition discards masks with no erased data (RS passthrough would merely repeat the direct assembly); the second discards masks exceeding the ECC budget or the cap on E. The planner `SubsetMaskPlanner.Plan` builds a lazy stream of feasible masks by suspicion levels:

1. **Base** — all collision slots at once. Their versions are no longer iterated: the volumes are excluded and recovered by RS as a whole (a configuration none of the stage-1/2 attempts has seen). An infeasible base → an empty plan: residual collisions are not covered by the stage.
2. **Level 1** — the base plus a single exclusion of every present non-collision volume. The order is by suspicion: ascending head-confirmation count, ties broken pseudo-randomly (a Fisher–Yates shuffle on Mt19937 seeded from H5 — determinism across calls and independence from the packet arrival order).
3. **Levels 2..MaxExtraExclusionLevel** — combinations of the first (most suspicious) `ShortlistSize` candidates: pairs, triples, …; the per-level binomial coefficients saturate at `long.MaxValue`.

An attempt costs Init ~O(E²·N) + 32 × Process ~O(E·N) + SHA-256(N·64), where E is the total number of erased data volumes (missing + excluded); excluding an ECC volume does not affect E but reduces the ECC budget. Benchmark calibration (`RsRaid16Demo bench`): with E ≤ 32 even the largest field (T = 65,535) yields dozens of attempts per second; with E ≥ 128 a single attempt costs seconds. Hence the separate limits: the cap on E (`MaxErasedDataVolumes`) bounds the price of one attempt, while `MaxAttempts` / `TimeBudget` bound their number. Cancellation and the budget are checked between attempts; progress reads "volume subset search, attempt: k / estimate".
The level-1 guarantee: if exactly one volume outside the collision base is corrupted and the limits hold, the mask excluding it will be generated and the attempt recovers the truth. For levels ≥ 2 the analogous statement is conditional: every corrupted volume must make it into the shortlist, and their number must not exceed `MaxExtraExclusionLevel`.

13. Result Verification and Memory Hygiene (`TrimAndVerify`)

```
TrimAndVerify(buffer):
    1. result ← buffer[0..FileSize)          // the last volume was zero-padded —
                                             // the tail is dropped
    2. if SHA-256(result) = H3 → return result
    3. otherwise: clear buffer and result, return ⊥
```

Correctness theorem. If assembly returned a non-empty result, its SHA-256 equals the H3 of the header accepted before the assembly started. Given the cryptographic strength of SHA-256, this means a bit-for-bit match with the original file. The converse does not hold: a refusal does not prove the data is missing (the budget may have run out), but an incorrect file is never produced.

Clearing the buffers on failure is memory hygiene: the content of a failed attempt (possibly containing a forgery) does not linger in the heap longer than necessary.

14. Slot Metrics and Observability

Member	Meaning
`HeaderReceptionCount`	header copies (saturating at int.MaxValue)
`ReceivedSectorCount`	numbers with ≥ 1 version
`ReceivedSectorCopyCount`	all sector copies (the sum of version counters)
`CollisionSectorCount`	numbers with > 1 version
`Coverage`	the share of received numbers of T, %
`BuildValidityMap`	`bool[T]`: number received
`FormatValidityMap`	`'█'` received / `'▓'` collision / `'░'` missing
`BuildCollisionMap`	number → collision multiplicity (only > 1 version)
`GetSectorVersions(n)`	a version snapshot with payload copies in preference order

Assembly progress is throttled to whole percents and never exceeds 99 before the actual completion; a successful outcome is reported as 100 (`AssemblyFinished`). Attempts inside the search publish no progress — the counter walks combinations/states/masks ("exhaustive search: k / C", "volume subset search: k / estimate").

15. Parameters and Limits (`SectorVersionSearchOptions`)

Parameter	Default	Meaning
`MaxExhaustiveCombinations`	100,000	the hard limit of C for exhaustive search
`MaxHeuristicAttempts`	100,000	the rotation-state limit (including the zeroth)
`TimeBudget`	30 s	a soft budget; checked between attempts

Stage 3 (section 12) is configured through the `SubsetSearch` group (`VolumeSubsetSearchOptions`):

`SubsetSearch` parameter	Default	Meaning
`MaxAttempts`	100,000	the attempt ceiling (fires together with the budget — whichever first)
`TimeBudget`	30 s	the stage's soft budget
`MaxErasedDataVolumes`	32	the cap on E — erased data volumes per attempt
`ShortlistSize`	64	the shortlist of suspicious volumes for levels t ≥ 2
`MaxExtraExclusionLevel`	3	the maximum size of extra exclusions

Overflow control: `CountCombinations` saturates at `long.MaxValue`; `CountRotationStates` caps the LCM at the attempt limit (product overflow is controlled via `LCM(a,b) = a / gcd(a,b) · b` with a divisor check); the per-level binomial coefficients saturate at `long.MaxValue`.

16. Worked Examples

16.1. Collision-free reception

A 100-byte file, ECC 0%: `N = 2, M = 0, T = 2`. Both sectors received (once each). `points = ∅` → a single attempt: `selected = [v₀, v₁]`, direct assembly `128 bytes → trim(100) → SHA-256 = H3` → success.

16.2. One choice point, `T₁ = 2`

Sector 0 has versions `A (c=3)` and `B (c=3)`. `C = 2`, timer `t₀`. Since `2 ≤ 100,000` and `2·t₀ ≤ 30 s` → exhaustive: attempt #0 — `A`, attempt #1 (odometer `(0) → (1)`) — `B`. The truth is found by the second attempt at the latest.

16.3. Two points, `T = (3, 2)`

`C = 6`, LCM = 6. Exhaustive search and rotation yield the **same** set of attempts (in a different order): the odometer walks `(0,0) (0,1) (1,0) (1,1) (2,0) (2,1)`, rotation walks the diagonals `A+X → B+Y → C+X → A+Y → B+X → C+Y`. Both are complete.

16.4. Three points, `T = (2, 2, 2)`

`C = 8` does not exceed the limit, but suppose the time estimate failed exhaustive search → rotation: LCM = 2, only `(A, X, P)` and `(B, Y, Q)` are checked. The six "mixed" combinations go unchecked — if the truth is among them, assembly honestly refuses (within the heuristic's budget). That is the price of scalability: a full pass would cost 4× more.

16.5. A winning forgery

Sector 5: truth `A (c=2)`, forgery `B (c=5)`. The list head is the single version `B` (the maximum counter is unshared) → no choice point, the selection always takes `B`, the final SHA-256 mismatches, refusal. No search can help — `A` is outside the search space (section 8). The system-level remedy: keep receiving; if repeats raise `A` to a tie, the next assembly call will see a choice point.

16.6. A silently corrupted volume (stage 3)

`N = 64`, `M = 8` (an ECC budget of 8 volumes). Volume 7 is corrupted, but its number is intact and D3 matches; the version is the only one. Stages 1–2: no choice points, the single attempt feeds RS the full map in which the corrupted volume is the truth; H3 mismatches. Stage 3: the base is empty (no collisions), level 1 excludes the present volumes one by one; on the mask `{7}` E = 1 ≤ 8 — RS recovers the truth, the hash matches. A single corruption is covered by level 1 guaranteed (within the limits): the level's masks do not depend on which particular volume is corrupted.

17. Summary of Guarantees

Provable properties:

1. **Result correctness.** A non-empty outcome ⟹ SHA-256 = H3 ⟹ (given SHA-256 strength) a bit-for-bit original.
2. **Direct-path completeness.** All data volumes received unambiguously ⟹ success on the very first attempt.
3. **Exhaustive-search completeness.** `C ≤ 100,000` and the time estimate within budget ⟹ the correct combination within the heads `T_j` will be found (short of cancellation/budget — then an honest refusal).
4. **Rotation equivalence for pairwise-coprime `T_j`.** LCM = C, all combinations reached.
5. **No false results.** Every path ends with either a proven file or `null`.

Heuristic properties (no guarantees):

6. Rotation coverage with common divisors — only the LCM diagonals.
7. Minority versions lie outside the current call's search space.
8. Fitting into `TimeBudget` is empirical, from the zero attempt's timing.
9. A single silent corruption outside the collision base is covered by stage-3 level 1 — given the ECC budget, the cap on E, and the limits (otherwise an honest refusal).
10. Multiple silent corruptions — only if every corrupted volume made it into the shortlist and their number does not exceed `MaxExtraExclusionLevel`.

Numeric summary: sector hash 72 bits (forgery ~2^{-72}), header hash 192 bits, the arbiter — SHA-256; exhaustive search ≤ 100,000 combinations, rotation ≤ 100,000 states, subset search ≤ 100,000 masks with the cap on E = 32; budgets of 30 s each (configurable).